# Dell Automation Platform 1.*x*

Pre-deployment Guide

August 2025
Rev. A00

DELL Technologies

## Notes, cautions, and warnings

**NOTE:** A NOTE indicates important information that helps you make better use of your product.

**CAUTION: A CAUTION indicates either potential damage to hardware or loss of data and tells you how to avoid the problem.**

**WARNING: A WARNING indicates a potential for property damage, personal injury, or death.**

Copyright © 2025 Dell Inc. All Rights Reserved. Dell Technologies, Dell, and other trademarks are trademarks of Dell Inc. or its subsidiaries. Other trademarks may be trademarks of their respective owners.

# Contents

**Chapter 1: Introduction** ..... **4**
About this document ..... 4
Audience ..... 4
Revision history ..... 4

**Chapter 2: VMware-based deployment requirements** ..... **5**
Prerequisites and limitations for OVA appliance deployment ..... 5
Proxy prerequisites and limitations ..... 7

**Chapter 3: Kubernetes-based deployment requirements** ..... **8**
Kubernetes-based deployment software requirements ..... 8
Kubernetes-based deployment environment requirements ..... 9
Prepare Kubernetes for an on-premises Dell Automation Platform deployment ..... 12
Initializing the Kubernetes cluster ..... 16
Initialize a single-node Kubernetes cluster ..... 16
Initialize a three-node Kubernetes cluster ..... 18
Configure required services and settings ..... 23
Optional configuration of a deployment VM ..... 26
Configure deployment host prerequisite software ..... 26

**Appendix A: Pre-deployment checklist** ..... **29**

# 1

# Introduction

**Topics:**


- About this document
- Audience
- Revision history


## About this document

This document describes the procedures that are used to prepare the customer environment for Dell Automation Platform deployment.

## Audience

This guide is intended for:

- Customers who are responsible for preparing their environments for Dell Automation Platform deployment using Kuberntes or VMware.
- Dell Technologies Professional Services personnel who are responsible for installing Dell Automation Platform in the customer environments.

## Revision history

**Table 1. Revision history**

| Date | Document revision | Description of changes |
|---|---|---|
| 08-27-2025 | A00 | Initial release. Some content is merged from Dell NativeEdge Orchestrator 3.1 Deployment Guide. |

2

# VMware-based deployment requirements

**NOTE:** Dell Technologies expects customers to install their VMware vCenter and ESXi environments, and ensure compliance with the requirements in this document, before Dell Automation Platform deployment. Customers who want assistance to perform those tasks should contact their Dell sales representative.

**Topics:**


- Prerequisites and limitations for OVA appliance deployment
- Proxy prerequisites and limitations


## Prerequisites and limitations for OVA appliance deployment

Note the following prerequisites and limitations when deploying Dell Automation Platform as an OVA appliance.

### Prerequisites for OVA appliance deployment

The customer supplies the ESXi cluster and VMware vCenter environment. The following prerequisites for Dell Automation Platform must be met.

| | |
|---|---|
| **CPU** | 32 CPU cores |
| **Memory** | 64 GB RAM |
| **Hard disk** | 2 TB local disk space provisioned |

### FQDNs for OVA appliance deployment

When configuring the FQDNs required to support the Dell Automation Platform portal and orchestrator, ensure the following:

- Both FQDNs must have the same base domain
- Both FQDNs must resolve to the same VM management IP address
- The FQDNs must not use `.local` in their addresses

**Table 2. Example FQDN configuration**

| Function | IP address | DNS | Note |
|---|---|---|---|
| Host | 192.168.0.50 | host.domain.edge | Server that hosts the deployment |
| Portal UI/API | 192.168.0.50 | portal.host.domain.edge | UI/API services derived from load balancer IP address |
| Orchestrator UI/API | 192.168.0.50 | orchestrator.host.domain.edge | UI/API services derived from load balancer IP address |
| mtls | 192.168.0.50 | mtls-orchestrator.host.domain.edge | Used by Dell Automation Platform infrastructure for mtls connection exposed from load balancer IP address |
| mtls-recovery | 192.168.0.50 | mtls-recovery-orchestrator.host.domain.edge | Used by Dell Automation Platform infrastructure for mtls recovery connection exposed from load balancer IP address |

**Table 2. Example FQDN configuration (continued)**

| Function | IP address | DNS | Note |
|---|---|---|---|
| Rendezvous | 192.168.0.50 | rv.local.fdo | Used by Dell Automation Platform infrastructure for secure device onboarding. For FDO in iDRAC PowerEdge systems, use the local rendezvous server address. For details, see the *Dell Automation Platform 1.x Administration Guide* on the Dell support site |
| | | rv.local.edge | Used in environments that include legacy NativeEdge devices |
| k8s-node1 | 192.168.0.50 | dap-k8s-1.domain.edge | IP address for Node1 of k8s cluster |
| Container Registry | 192.168.0.100 | registry.domain.edge | OCI registry for Dell Automation Platform container images |

# DNS records for OVA appliance deployment

The DNS must be configured to resolve both portal and orchestrator FQDNs, with the orchestrator using a wildcard format such as *.orchestrator.dap.base_domain, and DNS resolution must be verified using nslookup for the portal, orchestrator, and their mtls variants.

When configuring DNS, ensure the following:

- The DNS must contain the records for portal and orchestrator FQDNs, which means the DNS must be able to resolve these two FQDNs.
- The orchestrator FQDN should be in the wildcard format. For example: `*.dap.domain.tld`.
- If wildcard entries do not resolve, ensure they are added manually to the relevant DNS servers.
- If you do a DNS deployment with an OVA and you don't have a wildcard set up in your DNS environment, you must have an entry for mtls-*<orchestrator_fqdn>*.

In this case, two DNS records that are added to the DNS server should be as in the following table.

**Table 3. Example DNS records**

| Component | Domain name | VM IP address |
|---|---|---|
| Portal | `portal.dap.domain.tld` | `20.10.26.196` |
| Orchestrator | `*.dap.domain.tld` | `20.10.26.196` |

DNS verification:

When testing DNS, test resolving against the following FQDNs to ensure proper name resolution and connectivity:

```
nslookup <portal_fqdn>
nslookup <orchestrator_fqdn>
nslookup mtls-<orchestrator_fqdn>
nslookup mtls-recovery-<orchestrator_fqdn>
```

# Unconnected environment for OVA appliance deployment

For unconnected environments, ensure that the DNS is reachable within the network that the application is being installed in. Otherwise, the FQDN fails to resolve.

# OVA appliance deployment limitations

Snapshots:

**CAUTION: When taking a snapshot of the Dell Automation Platform virtual appliance, do not choose the *Include virtual machine's memory* option. Doing so causes Dell Automation Platform to fail to fully start.**

# Proxy prerequisites and limitations

Note the following SaaS orchestrator and portal prerequisites and limitations when deploying the Dell orchestrator proxy.

The customer supplies the ESXi cluster and VMware vCenter environment.

The following limitations apply:

| | |
|---|---|
| **NIC** | One NIC only |
| **VLAN** | No configured VLAN |
| **IPv6** | No IPv6 support |

**CAUTION: When taking a snapshot of the Dell Automation Platform virtual appliance, do not choose the *Include virtual machine's memory* option. Doing so causes Dell Automation Platform to fail to fully start.**

## Firewall policy

The supplied network should allow traffic for the following ports:

**Table 4. Firewall policy requirements**

| Port number | Traffic types | Policy | Description |
|---|---|---|---|
| 80 | Incoming | Allow | Allow external access to the host's HTTP service. |
| 443 | Incoming | Allow | Allow external access to the host's HTTPS service. |

3

# Kubernetes-based deployment requirements

**NOTE:** Dell Technologies expects customers to install their Kubernetes environments, and ensure compliance with the requirements in this document, before Dell Automation Platform deployment. Customers who want assistance to perform those tasks should contact their Dell representative.

**Topics:**


- Kubernetes-based deployment software requirements
- Kubernetes-based deployment environment requirements
- Prepare Kubernetes for an on-premises Dell Automation Platform deployment
- Initializing the Kubernetes cluster
- Configure required services and settings
- Optional configuration of a deployment VM


## Kubernetes-based deployment software requirements

Dell Automation Platform requires specific Kubernetes services, container runtimes, networking and storage plugins, and system tools—including Kubernetes 1.29.x–1.32.x, containerd, Calico CNI, Longhorn, Helm, Docker, and Ubuntu Server LTS 22.04–24.04—for successful deployment in a Kubernetes cluster.

The following table lists the recommended software packages and tools.

**Table 5. Kubernetes-based deployment software requirements**

| Software Package | Version | Optional or required | Note |
|---|---|---|---|
| Kubernetes (K8s) | 1.29.x–1.32.x (1.32.x recommended) | Required | Both client and server. Includes corresponding tools such as kubeadm. |
| containerd | 1.7.21 or later | Required | Container runtime for Kubernetes |
| runc | 1.1.14 or later | Required | Container runtime for Kubernetes |
| Linux CNI plugin | 1.5.1 or later | Required | Container Network Interface plugins |
| kube-vip | 0.3.8 or later | Optional | Only required for three-node Kubernetes deployment |
| Calico CNI | 3.28.0 or later | Required | Qualified Container Network Interface for Dell Automation Platform |
| Longhorn | 1.6.2 or later | Required | Qualified Block Storage Provider for Dell Automation Platform |
| haproxy | 3.0.12 | Required | Qualified Layer 7 ingress controller service for Dell Automation Platform |
| metalLB | 0.14.8 or later | Required | Qualified Layer 2 and Layer 3 load balancer service for Dell Automation Platform |
| Helm | 3.11.x–3.12.x | Required | K8s application manager for Dell Automation Platform deployments |
| Docker | 25.0.0 or later | Required | Docker engine and client tools, which the Dell Automation Platform orchestrator deployment package uses |
| Unzip | Latest | Required | Tool used to extract the Dell Automation Platform orchestrator installation bundle .zip |

**Table 5. Kubernetes-based deployment software requirements (continued)**

| Software Package | Version | Optional or required | Note |
|---|---|---|---|
| Ubuntu Server LTS | 22.04–24.04 | Required | Qualified host operating system for deploying Dell Automation Platform orchestrator.<br>**NOTE:**<br>Dell supports only this operating system. Other operating systems are not supported. |
| jq | Latest | Required | Lightweight JSON processor for Kubernetes |

**NOTE:** Kubernetes, containderd, runc, and CNI plugins may have compatibility restrictions depending on software versions. Be sure to check with the open-source vendor official documentation.

# Kubernetes-based deployment environment requirements

To deploy Dell Automation Platform orchestrator in a Kubernetes cluster, the environment must support PersistentVolumes and LoadBalancer services, and have a persistent container registry, internet access, and a deployment host with specific software tools and storage requirements. This section provides instructions and recommendations for deploying Dell Automation Platform orchestrator onto both single-node and three-node Kubernetes clusters, and it includes examples for installation and configuration of Kubernetes.

The Dell Automation Platform orchestrator deployment process requires common infrastructure for a Kubernetes environment. The following list provides general guidelines:

- Kubernetes node hosts that are deployed as virtual machines or on dedicated hardware
- A persistent container registry that follows OCI compliance, and is configured with TLS for storing Dell Automation Platform container images
- Internet access for pulling common open-source packages and tools that Kubernetes and their host operating systems use

**NOTE:** If Internet access is not available where Dell Automation Platform is deployed, consider preparing local repositories for all resources that are described in this document. Instructions for unconnected installation are not provided in this document.

The resource requirements for the clusters that are dedicated for the Dell Automation Platform orchestrator are detailed in the following lists. For specific software version recommendations, see Kubernetes-based deployment software requirements.

## Optional deployment host for Kubernetes-based deployment environments

- Optional host used to manage the Kubernetes cluster and deploy Dell Automation Platform
- Access to the target Kubernetes cluster
- Software tools: Docker, Zip, kubectl
- At least 30 GB of storage to download and extract the Dell Automation Platform bundle

**NOTE:** If a deployment host is unavailable, Dell Automation Platform orchestrator may be deployed directly from the Kubernetes node.

## FQDN guidance for Kubernetes-based deployment environments

The use of .local as a FQDN label is not supported and causes issues when attempting to onboard assets. For more information, see the related Knowledge Base Article.

# DNS recommendations for Kubernetes-based deployment environments

The DNS is an essential component of modern Kubernetes-based applications. It enables using FQDN for all hosts, nodes, and application services. An FQDN schema is recommended when deploying Dell Automation Platform orchestrator. In the absence of DNS services, manual hostname resolution can be used.

For the infrastructure and orchestrator to resolve the required DNS records for proper connectivity, Dell Technologies recommends the following:

- Onboarding Dell Automation Platform infrastructure
  - When global rendezvous is enabled, the Dell Automation Platform orchestrator and Dell Automation Platform infrastructure must be able to resolve rv.dell.com
  - When Internet access is unavailable, or local rendezvous service is preferred, ensure that the Dell Automation Platform orchestrator IP resolves to one of the following DNS host records: rv.local.edge, rv1.local.edge, ..., rv20.local.edge. For details, see the *Dell Automation Platform 1.x Administration Guide*
- Kubernetes clusters
  - Dell Technologies recommends that all nodes, Kube-VIPs, and load balancer IPs are resolvable, where applicable
  - When considering the FQDN schema, the load balancer IP is used for the Dell Automation Platform orchestrator UI and API services. The Kube-VIP ensures highly available management of the Kubernetes control-plane nodes
- Dell Automation Platform UI, API, and services
  - The load balancer should be configured with a single IP, which is used for Dell Automation Platform orchestrator UI, API, and services
  - When deploying with the FQDN, additional mtls- and mtls-recovery DNS host records are necessary for Dell Automation Platform infrastructure connectivity to the Dell Automation Platform orchestrator

If DNS services are not available in the environment, local host entries are required on each Kubernetes node and the optional deployment host.

If the DNS records contain wildcards and the wildcard entries do not resolve, ensure they are added manually to the relevant DNS servers.

An example single-node DNS schema is provided in the following table:

**Table 6. Example single-node DNS schema**

| Function | IP address | DNS | Note |
|---|---|---|---|
| Host | 192.168.0.50 | host.domain.edge | Server that hosts the deployment |
| Portal UI/API | 192.168.0.50 | portal.host.domain.edge | UI/API services derived from load balancer IP address |
| Orchestrator UI/API | 192.168.0.50 | orchestrator.host.domain.edge | UI/API services derived from load balancer IP address |
| mtls | 192.168.0.50 | mtls-orchestrator.host.domain.edge | Used by Dell Automation Platform infrastructure for mtls connection exposed from load balancer IP address |
| mtls-recovery | 192.168.0.50 | mtls-recovery-orchestrator.host.domain.edge | Used by Dell Automation Platform infrastructure for mtls recovery connection exposed from load balancer IP address |
| Rendezvous | 192.168.0.50 | rv.local.fdo | Used by Dell Automation Platform infrastructure for secure device onboarding. For FDO in iDRAC PowerEdge systems, use the local rendezvous server address. For details, see the *Dell Automation Platform 1.x Administration Guide* |
| | | rv.local.edge | Used in environments that include legacy NativeEdge devices |
| k8s-node1 | 192.168.0.50 | dap-k8s-1.domain.edge | IP address for Node1 of k8s cluster |
| Container Registry | 192.168.0.100 | registry.domain.edge | OCI registry for Dell Automation Platform container images |

The total required IP addresss for a single-node Kubernetes cluster running Dell Automation Platform orchestrator is one.

An example three-node DNS schema is provided in the following table:

**Table 7. Example three-node DNS schema**

| Function | IP address | DNS | Note |
|---|---|---|---|
| Host | 192.168.0.50 | host.domain.edge | Server that hosts the deployment |
| Portal UI/API | 192.168.0.50 | portal.host.domain.edge | UI/API services derived from load balancer IP address |
| Orchestrator UI/API | 192.168.0.50 | orchestrator.host.domain.edge | UI/API services derived from load balancer IP address |
| mtls | 192.168.0.50 | mtls-orchestrator.host.domain.edge | Used by Dell Automation Platform infrastructure for secure mtls |
| mtls-recovery | 192.168.0.50 | mtls-recovery-orchestrator.host.domain.edge | Used by Dell Automation Platform infrastructure for mtls recovery |
| Rendezvous | 192.168.0.50 | rv.fdo.edge | Used by Dell Automation Platform infrastructure for secure device onboarding. For FDO in iDRAC PowerEdge systems, use the local rendezvous server address. For details, see the *Dell Automation Platform 1.x Administration Guide* on the Dell support site |
| | | rv.local.edge | Used in environments that include previously added NativeEdge devices |
| k8s-node1 | 192.168.0.51 | dap-k8s-1.domain.edge | IP address for node1 of k8s cluster |
| k8s-node2 | 192.168.0.52 | dap-k8s-2.domain.edge | IP address for node2 of k8s cluster |
| k8s-node3 | 192.168.0.53 | dap-k8s-3.domain.edge | IP address for node3 of k8s cluster |
| Kube-VIP | 192.168.0.54 | dap-k8s-vip.domain.edge | Virtual IP address for multi-node cluster management |
| Container Registry | 192.168.0.100 | registry.domain.edge | OCI registry for Dell Automation Platform container images |

The total required IP addresses for a three-node Kubernetes cluster running Dell Automation Platform orchestrator is five.

**CAUTION: When provisioning Dell Automation Platform orchestrator, the orchestrator UI/API, mtls, and mtl-recovery must be the same load balancer IP address.**

**NOTE:** The orchestrator portal IP address is used to access the user interface. If the orchestrator portal IP address is specified instead of FQDN, when the deployment script is run, Dell Automation Platform infrastructure devices default to using an internal host mapping of mtls and mtls-recovery.

**NOTE:** DNS schemas can include numerous rendezvous entries. For details, see the *Dell Automation Platform 1.x Administration Guide* on the Dell support site.

**NOTE:** The load balancer and node IP address can be the same if wanted when provisioning a single-node Kubernetes cluster. Using a unique IP address for the load balancer is also supported.

**NOTE:** Kube-VIP is not required for single-node Kubernetes clusters.

## Network ports used by Kubernetes-based deployment environments

The following table provides common TCP ports that Kubernetes leverages.

**Table 8. TCP ports**

| Protocol | Direction | Port range | Purpose | Used by |
|---|---|---|---|---|
| TCP | Inbound | 6443 | K8s API server | All clients and nodes accessing the K8s cluster API |
| TCP | Inbound | 2379–2380 | Etcd | Kube-api server etcd service |
| TCP | Inbound | 10250 | Kubelet API | K8s control-plane nodes |
| TCP | Inbound | 10259 | Kube-scheduler | K8s nodes |
| TCP | Inbound | 10257 | Kube-control-manager | K8s nodes |
| TCP | Inbound | 10256 | Kube-proxy | K8s nodes and load balancer service |
| TCP | Inbound | 30000–32767 | NodePort | If configured, optional node ports that deployed Kubernetes services can leverage |

For more information about Dell Automation Platform security and networking, including necessary ports, see the *Dell Automation Platform 1.x Security Configuration Guide* on the Dell support site.

## NTP recommendations

Configure NTP to ensure that each node on the cluster is configured with the same and accurate NTP servers. This step is crucial for keeping each pod in the Kubernetes cluster synchronized. It is recommended that you verify the IP address and FQDN of the NTP server for your environment before configuring Kubernetes. If no local NTP is available, you can leverage a public Internet NTP server.

# Prepare Kubernetes for an on-premises Dell Automation Platform deployment

Follow this procedure to prepare Kubernetes for Dell Automation Platform deployment, which installs both the Dell Automation Platform portal and the Dell Automation Platform orchestrator. These instructions do not include security hardening. You must follow your company internal security policies. To perform these steps, you need admin rights. In the case of an existing cluster, you must consult your system administrator before proceeding.

### Prerequisites

Perform these instructions before Dell Technologies personnel arrive onsite.

These instructions leverage common operating systems, open-source tools, packages, and configurations recommended for Dell Automation Platform orchestrator including:

- Kubernetes nodes: Each node in this example is a virtual machine running as an Ubuntu 24.04 live server with a Static IP and resolvable with DNS.
- Kernel parameters and modules: fs.inotify, net.bridge, net.ipv4, overlay, br_netfilter
- Container runtimes: containerd, runc, and CNI plugins
- Deployment tools: Helm, Docker
- Kubernetes management and runtimes: kubectl, kubeadm, kubelet
- Container network: Calico CNI
- Ingress controller: haproxy
- Load balancer: MetalLB
- Block storage provider: Longhorn

Each Virtual Machine in this example uses the following resources:

- Operating system: Ubuntu Server 22.04–24.04.1 LTS
- CPU: 16-core vCPU
- Memory: 32 GB for both Dell Automation Platform portal and Dell Automation Platform orchestrator
- Disk: 1 TB

### About this task

The following figure illustrates the DAP on-premises deployment architecture:

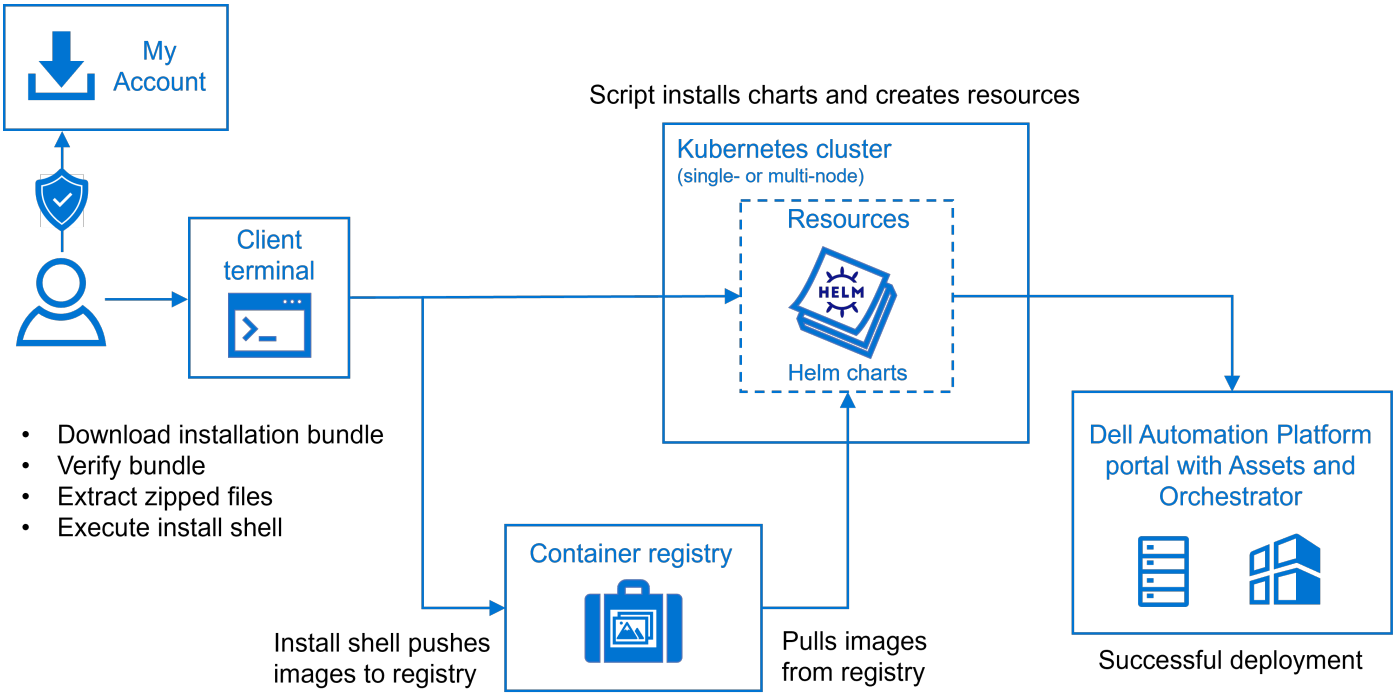


**Figure 1. DAP on-premises deployment architecture**

This section contains an example that provides instructions for preparing and installing a Kubernetes cluster that is used for deploying the Dell Automation Platform orchestrator. The instructions include preparing Kubernetes as a single-node K8s cluster, or configuring Kubernetes as a three-node cluster. Each node is configured as an untainted control-plane, allowing workloads and service management across all nodes of the cluster.

Perform the following steps on each node host of the cluster.

### Steps

1. Update the system and install the prerequisite packages using the integrated `apt-get` package manager, then configure NTP.
   a. Update the system and install packages:

```
sudo apt update
sudo apt install apt-transport-https ntp ntpdate
sudo apt upgrade
```

> **NOTE:** The result of the `apt upgrade` command may recommend that the system is rebooted to finish the updates. Reboot the system as needed before continuing.

   b. Configure NTP services:

```
sudo systemd-timesyncd
sudo systemctl status systemd-timesyncd # Check if systemd-timesyncd is running on
the node. If running, no action required.
sudo nano /etc/systemd/timesyncd.conf
[Time]
NTP=ntp.ubuntu.com
FallbackNTP=ntp.ubuntu.com

sudo systemctl restart systemd-timesyncd # Restart the service to apply changes
sudo timedatectl set-ntp true
timedatectl status
```

```
sudo systemctl enable systemd-timesyncd
sudo systemctl start systemd-timesyncd
```

2. Load the overlay and `br_netfilter` kernel modules and set `sysctl` parameters that the Dell Automation Platform orchestrator and Kubernetes use. Overlay provides the file system overlay functionality that Kubernetes pods rely on for their abstraction. The br_netfilter module enables bridge get filter support for the kernel, which is leveraged by Kubernetes and its network overlay to function correctly. The Dell Automation Platform orchestrator requires the `fs.inotify.max_user_instances` and `fs.inotify.max_user_watches` settings to raise the limit of the number of processes that can watch files.
   a. Add modules:

   ```
   echo -e "overlay\nbr_netfilter" | sudo tee /etc/modules-load.d/containerd.conf
   sudo modprobe -a overlay br_netfilter
   ```

   b. Verify that the modules are loaded:

   ```
   lsmod |egrep 'overlay|br_netfilter'
   ```

   The resulting output should look similar to the following:

   ```
   br_netfilter           32768  0
   bridge                307200  1 br_netfilter
   overlay               151552  0
   ```

3. Create `sysctl` configuration files for Dell Automation Platform and Kubernetes:

   ```
   echo -e "fs.inotify.max_user_watches = 1048576\nfs.inotify.max_user_instances = 1024"
   | sudo tee /etc/sysctl.d/10-orchestrator.conf
   echo -e "net.bridge.bridge-nf-call-ip6tables = 1\nnet.bridge.bridge-nf-call-iptables
   = 1\nnet.ipv4.ip_forward = 1" | sudo tee /etc/sysctl.d/10-kubernetes.conf
   ```

4. Apply the `sysctl` configurations:

   ```
   sudo sysctl --system
   ```

5. Download the container registry certificate, verify, and add to the certificate store. Each node in the Kubernetes cluster uses the container registry certificate to authenticate with the registry and pull Dell Automation Platform container images.
   a. Use `openssl` to download the certificate of the registry:

   ```
   echo | openssl s_client -showcerts -connect <registryURL>:<port> 2>/dev/null |
   openssl x509 > registry.crt
   ```

   b. Verify that the certificate is valid:

   ```
   openssl x509 -in registry.crt -text -noout
   ```

   c. Copy the certificate to the local certificate store and update:

   ```
   sudo cp registry.crt /usr/local/share/ca-certificates/
   sudo update-ca-certificates
   ```

6. Disable swap space. While swap space is generally recommended in Linux environments, Kubernetes cannot accurately account for swap memory usage. For optimal scheduler performance and system stability, swap must be disabled. Leaving it enabled may result in unpredictable behavior and performance issues.

   ```
   sudo swapoff -a
   sudo sed -i -e '/swap/d' /etc/fstab
   ```

7. Install containerd, runc, and CNI plugins. Containerd provides a runtime for Kubernetes to manage containers, and runc is a low-level container runtime that the containerd service leverages. The CNI plugins provide an interface for the cluster to leverage network capabilities such as bridge, IPvlan, loop back, MAC, VLAN, and other network components.

a. Install runc:

```
curl -fsSLo runc.amd64 https://github.com/opencontainers/runc/releases/download/
v1.1.14/runc.amd64
sudo install -m 755 runc.amd64 /usr/local/sbin/runc
```

b. Create the containerd configuration file to allow CNI plugins:

```
sudo mkdir /etc/containerd
sudo vi /etc/containerd/config.toml
version = 2
[plugins]
  [plugins."io.containerd.grpc.v1.cri"]
    sandbox_image = "registry.k8s.io/pause:3.9"
  [plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc]
    runtime_type = "io.containerd.runc.v2"
    [plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc.options]
      SystemdCgroup = true
```

c. Install containerd:

```
curl -fLo containerd-1.7.21-linux-amd64.tar.gz https://github.com/containerd/
containerd/releases/download/v1.7.21/containerd-1.7.21-linux-amd64.tar.gz
sudo tar Cxzf /usr/local containerd-1.7.21-linux-amd64.tar.gz
```

d. Install containerd service and verify that the service is running:

```
sudo curl -fsSLo /etc/systemd/system/containerd.service https://
raw.githubusercontent.com/containerd/containerd/main/containerd.service
sudo sed -i '/^LimitCORE=infinity$/a LimitNOFILE=infinity' /etc/systemd/system/
containerd.service
sudo systemctl daemon-reload && sudo systemctl enable --now containerd
sudo systemctl status containerd
```

e. Install CNI network plugins:

```
curl -fLo cni-plugins-linux-amd64-v1.5.1.tgz https://github.com/containernetworking/
plugins/releases/download/v1.5.1/cni-plugins-linux-amd64-v1.5.1.tgz
sudo mkdir -p /opt/cni/bin
sudo tar Cxzvf /opt/cni/bin cni-plugins-linux-amd64-v1.5.1.tgz
```

8. Install Kubernetes runtime and tools. Kubelet is the main agent that runs on a Kubernetes node. It is responsible for registering the node with the API server, receiving requests from the control plane to run pods, and ensuring that pods are running in specific configurations while coordinating with the container engine. Kubectl is the Kubernetes API client that allows the user to manage the cluster. Kubeadm is a tool that is used for initializing the Kubernetes cluster and configuring Kubelet.

**NOTE:** For more information, see Intalling kubeadm from the Kubernetes website.

a. Add the Kubernetes release key and update apt repositories:

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo gpg --
dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
pkgs.k8s.io/core:/stable:/v1.30/deb/ /' | sudo tee /etc/apt/sources.list.d/
kubernetes.list
sudo apt update
```

**NOTE:** If the directory `/etc/apt/keyrings` does not exist, then use the following command to create it before running the `curl` command above.

```
# sudo mkdir -p -m 755 /etc/apt/keyrings
```

b. Install Kubernetes tools, and mark them to hold upgrades. It is recommended to hold upgrades so that no accidental apt updates disrupt the Kubernetes cluster.

```
sudo apt install kubeadm kubelet kubectl
sudo apt-mark hold kubeadm kubelet kubectl
```

9. Install the Helm package management tool. Helm is leveraged to install the Longhorn block storage provide and is also used during the Dell Automation Platform deployment.

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/
get-helm-3
chmod 700 get_helm.sh
sudo ./get_helm.sh --version v3.12.0
```

10. Optionally, install the Docker Engine and client tools. If deploying Dell Automation Platform orchestrator directly on the Kubernetes node, the Docker Engine and client tools are used in the deployment process. The following commands install Docker using the official installation script that Docker maintains. Dell Technologies recommends using the script as it does not overwrite the existing containerd configuration from the previous steps.

```
sudo curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

11. Add the local user to the Docker group (optional):

```
sudo usermod -aG docker $USER
```

#### Next steps

For single-node Kubernetes, continue to the next step. For three-node Kubernetes, repeat all steps on each of the two additional node hosts before continuing.

# Initializing the Kubernetes cluster

Users must initialize the cluster before deployment. The following sections describe this task for single-node and three-node clusters.

## Initialize a single-node Kubernetes cluster

To initialize a single-node Kubernetes cluster, create a `kubeadm` configuration file and use `kubeadm` to initialize the cluster. The cluster is then ready for deploying a pod network and further configuration.

#### About this task

Perform the following steps for a single-node cluster. For three-node clusters, go to Initialize a three-node Kubernetes cluster.

#### Steps

1. Verify that the Kubernetes node and registry are resolvable on the network. For a single-node Kubernetes deployment, it is only required to add the registry.

   If DNS is leveraged, use the `nslookup` command to verify that IP and hostname are resolvable on the Kubernetes node and deployment VM (if applicable):

   ```
   nslookup dap-k8s-1.mydomain.edge
   nslookup registry.mydomain.edge
   ```

   If DNS is not leveraged, update the `/etc/hosts` on the Kubernetes node and deployment VM (if applicable):

   ```
   sudo vi /etc/hosts
   192.168.0.50    dap-k8s-1.mydomain.edge
   192.168.0.100    registry.mydomain.edge
   ```

**NOTE:** These hostnames and URLS are examples only. Update according to your environment.

2. Create a `kubeadm` configuration file.

   This configuration file defines the basic cluster and kubelet configuration. The contents of this configuration may change, depending on your environment.

```
vi k8s-singlenode.yaml

apiVersion: kubeadm.k8s.io/v1beta3
kind: ClusterConfiguration
controlPlaneEndpoint: "dap-k8s-1.mydomain.edge:6443"
kubernetesVersion: v1.32.8
networking:
  podSubnet: 161.200.0.0/16
  serviceSubnet: 161.210.0.0/16
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
serverTLSBootstrap: true
maxPods: 180
```

   Use the following guidelines to make the necessary changes. Calico CNI leverages the `podsSubnet` and `serviceSubnet`. They can be changed to a wanted private subnet. When selecting a CNI subnet, ensure that the subnet range is not in use to prevent network overlap and IP crashes. The `maxPods` in the Kubelet configuration is set to 150 pods to account for the Dell Automation Platform orchestrator namespace and the additional services that are deployed in the Kubernetes cluster.

   - Change the `controlPlaneEndpoint` value to the node FQDN. If DNS is not leveraged, then replace the FQDN with hostname instead.
   - Change the `kubernetesVersion` value to the wanted version within the range specified in Kubernetes-based deployment software requirements.
   - Change the `podSubnet` and `serviceSubnet` to the wanted subnet ranges.

3. Use `kubeadm` to initialize the Kubernetes single-node cluster:

```
sudo kubeadm init --upload-certs --config=k8s-singlenode.yaml
```

   Upon successful initialization, the following message (or something similar) displays:

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

You can now join any number of the control-plane node running the following command
on each as root:

  kubeadm join k8s-node1:6443 --token kx9g5z.8nqra3kyhuzrx4us \
        --discovery-token-ca-cert-hash
sha256:4ab20fb7d3680bd25fff4cd2c32bee8ececc1e606c41707e606f72d7c5c2849d \
        --control-plane --certificate-key
acf2b9cc2cf85cf765128415768457f23f2684380f6113b84af7e18831f37791

Please note that the certificate-key gives access to cluster sensitive data, keep it
secret!
As a safeguard, uploaded-certs will be deleted in two hours; If necessary, you can use
"kubeadm init phase upload-certs --upload-certs" to reload certs afterward.
```

```
Then you can join any number of worker nodes by running the following on each as root:

kubeadm join k8s-node1:6443 --token kx9g5z.8nqra3kyhuzrx4us \
        --discovery-token-ca-cert-hash
sha256:4ab20fb7d3680bd25fff4cd2c32bee8ececc1e606c41707e606f72d7c5c2849d
```

4. Configure kubectl for root and local user:

```
mkdir -p $HOME/.kube
sudo mkdir /root/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
sudo cp -i /etc/kubernetes/admin.conf /root/.kube/config
```

5. Accept the certificate signing requests (CSRs) for any pending certificates:

```
kubectl get csr
kubectl certificate approve <cert_name>
```

**NOTE:** By default, the Kubernetes cluster periodically issues new certificate signing requests until all are approved. If there are several pending CSRs, use the following command to approve all CSRs at once:

```
kubectl get csr | grep Pending | awk '{print $1}' | xargs -I {} kubectl
certificate approve {}
```

6. Remove the taint on the control-plane node:

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

7. Verify that kubectl can send APIs to the cluster:

```
kubectl get nodes
kubectl get pods -A
```

**NOTE:** The status of the node and pods may show `NotRead` and `Pending` status. These statuses are expected at this time, as the Calico CNI has not yet been configured.

### Next steps

Continue to Configure required services and settings.

## Initialize a three-node Kubernetes cluster

Initialize a three-node Kubernetes cluster with kube-vip for high availability of the Kubernetes service. Ensure control plane redundancy and workload resilience with proper configuration.

### Prerequisites

This task requires setup of kube-vip, which requires an extra virtual IP (VIP) and additional configuration steps. See Example three-node DNS schema for reference.

### About this task

This task provides high availability of the Kubernetes service, but it does not provide high availability of Dell Automation Platform. When a node fails, the stateless service can be replicated in another node. If the stateful data has replication, for example, Longhorn, the data will move to the node as well. This process will result in downtime of the service.

For a three-node cluster, Dell Technologies recommends setting up the cluster as three control-plane nodes with a configured kube-vip. That configuration ensures that the cluster management API and other services are highly available. A three-node, high availability (HA) cluster offers these advantages:

| | |
|---|---|
| **Etcd quorum** | Etcd (the key-value store for Kubernetes) requires a majority (n/2 + 1) to function. In a three-node Etcd cluster, quorum is 2, so the cluster can tolerate one node failure and still operate. |
| **Control plane redundancy** | If all three nodes run control plane components, the cluster can survive the failure of one control plane node. Kubernetes components like the API server can be load-balanced across the nodes. |
| **Workload resilience** | If workloads are spread across all three nodes, and PodDisruptionBudgets and anti-affinity rules are used, the cluster can maintain application availability even if one node fails. |

With kube-vip set up in your three-node Kubernetes cluster, there is no requirement to manually identify the leader node. kube-vip automates the leader election process and manages the virtual IP for the control plane, ensuring high availability and load balancing without manual intervention.

Perform the following steps for a three-node cluster.

### Steps

1. Verify that the Kubernetes node and registry are resolvable on the network.

   If DNS is leveraged, use the `nslookup` command to verify that IP and hostname are resolvable on the Kubernetes node and deployment VM (if applicable):

   ```
   nslookup dap-k8s-1.mydomain.edge
   nslookup dap-k8s-2.mydomain.edge
   nslookup dap-k8s-3.mydomain.edge
   nslookup registry.mydomain.edge
   ```

   If DNS is not leveraged, update the `/etc/hosts` file on each Kubernetes node and deployment VM (if applicable):

   ```
   sudo vi /etc/hosts
   192.168.0.51    dap-k8s-1.mydomain.edge
   192.168.0.52    dap-k8s-2.mydomain.edge
   192.168.0.53    dap-k8s-3.mydomain.edge
   192.168.0.54    dap-k8s-vip.mydomain.edge
   192.168.0.100    registry.mydomain.edge
   ```

   **NOTE:** These hostnames and URLs are examples only. Update according to your environment.

   **NOTE:** The kube-vip does not initially resolve until the kube-vip static pod is deployed in later steps of this task.

2. Generate a kube-vip configuration YAML for a static pod.

   In this example, a kube-vip static pod is deployed into the Kubernetes cluster during cluster initialization. The kube-vip.io project recommends creating the manifest YAML leveraging a Docker container. This container may be run directly on the Kubernetes node or on a separate host, such as the deployment VM. If running on the Kubernetes node, ensure that the Docker engine and runtime are installed according to the instructions in step 10 of Prepare Kubernetes for an on-premises Dell Automation Platform deployment.

   a. Download kube-vip container image:

      Run `docker` with `sudo`—unless the user has been added to the docker group and the shell session has reloaded. For more information, see the instructions in step 11 of Prepare Kubernetes for an on-premises Dell Automation Platform deployment.

      ```
      docker pull ghcr.io/kube-vip/kube-vip:v0.8.3
      ```

   b. Generate a manifest file using the interface name of the Kubernetes host primary network interface and the wanted kube-vip IP. This IP must not already be in use.

      The kube-vip manifest YAML is leveraged on each Kubernetes node. Therefore it is crucial that each Kubernetes node in the cluster uses identical interface names for the primary interface. In Ubuntu Server 22.04 LTS, it is typically "ens33". It may vary depending on version or custom network configuration.

      ```
      docker run --network host --rm ghcr.io/kube-vip/kube-vip:v0.8.3 manifest pod --
      interface <Local_INTERFACE> --vip <VIP_ADDRESS> --controlplane --services --arp --
      leaderElection | sudo tee /etc/kubernetes/manifests/kube-vip.yaml
      ```

This command automatically outputs the manifest YAML to the required manifest directory in `'/etc/kubernetes/manifests/'`.

c. Upload the generated manifest YAML to each node in the cluster:

```
sudo scp /etc/kubernetes/manifests/kube-vip.yaml <user>@<node-1-hostname>:/home/<user>/
sudo scp /etc/kubernetes/manifests/kube-vip.yaml <user>@<node-2-hostname>:/home/<user>/
sudo scp /etc/kubernetes/manifests/kube-vip.yaml <user>@<node-3-hostname>:/home/<user>/
```

**NOTE:** Disregard *<node-1>* if you generated the kube-vip manifest from that host.

d. On each Kubernetes node, copy the uploaded kube-vip.yaml to the Kubernetes manifest directory:

```
sudo cp kube-vip.yaml /etc/kubernetes/manifests/kube-vip.yaml
```

3. Create a kubeadm configuration file for a three-node cluster.

This configuration file defines the basic cluster and kubelet configuration. The contents of this configuration may change, depending on your environment.

```
vi k8s-multinode.yaml

apiVersion: kubeadm.k8s.io/v1beta3
kind: ClusterConfiguration
controlPlaneEndpoint: "dap-k8s-vip.domain.edge:6443"
kubernetesVersion: v1.32.8
networking:
  podSubnet: 161.200.0.0/16
  serviceSubnet: 161.210.0.0/16
apiServer:
  certSANs:
  - "dap-k8s-1.mydomain.edge"
  - "dap-k8s-2.mydomain.edge"
  - "dap-k8s-3.mydomain.edge"
  - "dap-k8s-vip.mydomain.edge"
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
serverTLSBootstrap: true
maxPods: 180
```

Use the following guidelines to make the necessary changes. Calico CNI leverages the `podsSubnet` and `serviceSubnet`, and they can be changed to a wanted private subnet. When selecting a CNI subnet, ensure that the subnet range is not in use to prevent network overlap and IP crashes. The `maxPods` in the Kubelet configuration is set to 150 pods to account for the Dell Automation Platform orchestrator namespace and the additional services that are deployed in the Kubernetes cluster.

- Change the `controlPlaneEndpoint` value to the node FQDN. If the DNS is not leveraged, then replace the FQDN with hostname instead.
- Change the `kubernetesVersion` value to the wanted version within the range specified in Kubernetes-based deployment software requirements.
- Change the `podSubnet` and `serviceSubnet` to the wanted subnet ranges.
- Change the `certSANs:` to match the node and kube-vip FQDN. If the DNS is not leveraged, match them to the hostnames added to `/etc/hosts`.

4. Initialize the first control-plane node of the three-node Kubernetes cluster.

a. Initialize the first control-plan of the cluster with kubeadm:

```
sudo kubeadm init --upload-certs --config=k8s-multinode.yaml
```

Upon successful initialization, the following message or something similar displays:

```
Your Kubernetes control-plane has initialized successfully!
```

```
To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

You can now join any number of the control-plane node running the following command
on each as root:

  kubeadm join k8s-node1:6443 --token kx9g5z.8nqra3kyhuzrx4us \
        --discovery-token-ca-cert-hash
sha256:4ab20fb7d3680bd25fff4cd2c32bee8ececc1e606c41707e606f72d7c5c2849d \
        --control-plane --certificate-key
acf2b9cc2cf85cf765128415768457f23f2684380f6113b84af7e18831f37791

Please note that the certificate-key gives access to cluster sensitive data, keep
it secret!
As a safeguard, uploaded-certs will be deleted in two hours; If necessary, you can
use
"kubeadm init phase upload-certs --upload-certs" to reload certs afterward.

Then you can join any number of worker nodes by running the following on each as
root:

kubeadm join k8s-node1:6443 --token kx9g5z.8nqra3kyhuzrx4us \
        --discovery-token-ca-cert-hash
sha256:4ab20fb7d3680bd25fff4cd2c32bee8ececc1e606c41707e606f72d7c5c2849d
```

**NOTE:** Copy the `kubeadm join` command for joining a "control-plane" from the initialization output, as it is used in the following steps to join the additional two control-plane nodes.

5. Configure kubectl for root and local user:

```
mkdir -p $HOME/.kube
sudo mkdir /root/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
sudo cp -i /etc/kubernetes/admin.conf /root/.kube/config
```

6. Verify that kubectl can send APIs to the cluster:

```
kubectl get nodes
kubectl get pods -A
```

**NOTE:** The status of the node and pods may show `NotRead` and `Pending` status. These statuses are expected at this time, as the Calico CNI has not yet been configured.

7. Initialize and join the second and third control plane nodes of the cluster.
   a. On both the second and third Kubernetes node hosts, the kube-vip.yaml is already uploaded and copied to the manifest directory. Verify that the kube-vip.yaml is present on each host. If the file is missing, follow the steps that are previously mentioned in this task.

   ```
   sudo ls -al /etc/kubernetes/manifests/kube-vip.yaml
   ```

   b. Join each node as a control plane using the output from the kubeadm command from step 4.a in this task.

If the `kubeadm init` output is no longer available, a new join token can be generated using the following command on the first control plane node:

```
kubeadm token create --print-join-command --certificate-key $(kubeadm init phase
upload-certs --upload-certs)
```

c. Run the generated `kubeadm join` command, similar to the following example.

**NOTE:** The following command is an example output. Running the command, as copied from this document, does not successfully join your nodes to the cluster.

```
sudo kubeadm join k8s-node1:6443 --token na1dim.5s5r7q4ta1plwoli \
        --discovery-token-ca-cert-hash
sha256:0e5c0c697fe3b9c0f2d97dc4263223b1f0fad3079ebf62f4c6bcf72c4434545e \
        --control-plane --certificate-key
8e19ad7b7b18a168ecc85e6144cb0cdb1a66a8497ac39e317119f66e81692e82
```

The command output indicates that the node was successful in joining the cluster. For example:

```
This node has joined the cluster and a new control plane instance was created:
```

d. Repeat the `kubeadm join` command on the third Kubernetes node host.

e. Optional: If API management is wanted on each node of the Kubernetes cluster, configure kubectl for the local and root user on each node:

```
mkdir -p $HOME/.kube
sudo mkdir /root/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
sudo cp -i /etc/kubernetes/admin.conf /root/.kube/config
```

8. Finalize the initial Kubernetes setup for the three-node cluster.

Now that all three Kubernetes nodes have been initialized and joined to the cluster, each Kubernetes node has pending certificate signing requests (CSRs). These requests must be approved for each node to participate in the cluster. By default, Kubernetes applies a taint to each control-plane node that restricts scheduling of workloads on those nodes. Since each Kubernetes node in this configuration acts as both control-plane and worker, the taint is removed from each control-plane.

a. Accept the CSRs for any pending certificates:

```
kubectl get csr
kubectl certificate approve <cert_name>
```

**NOTE:** By default, the Kubernetes cluster periodically issues new CSRs until all are approved. If there are several pending CSRs, use this command to approve all at once: "`kubectl get csr | grep Pending | awk '{print $1}' | xargs -I {} kubectl certificate approve {}`".

b. Remove the control plane taint on all nodes:

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

c. Verify that kubectl can send APIs to the cluster:

```
kubectl get nodes
kubectl get pods -A
```

**NOTE:** The status of the node and pods may show `NotRead` and `Pending` status. These statuses are expected at this time, as the Calico CNI has not yet been configured.

# Configure required services and settings

### About this task

More services are configured than the Dell Automation Platform orchestrator requires. The following steps can be performed from a deployment host, or from a Kubernetes node in the cluster. In a three-node Kubernetes cluster, the following steps should only be performed on a single node. Do not repeat on each node. If using a deployment host, see Optional configuration of a deployment VM before continuing.

### Steps

1. Install Calico CNI using the manifest.

   Calico is a container network interface (CNI), which leverages the `podSubnet` and `serviceSubnet` values that are defined in the kubeadm configuration files.

   ```
   kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/
   manifests/calico.yaml
   ```

   **NOTE:** Calico containers are hosted on the Docker website. If the environment is rate pull limited by docker.io, then the containers may show `imagePullBackoff` errors. See the Article—Dell NativeEdge: How to resolve pull rate limits when trying to deploy Calico for NativeEdge Deployment for instructions on creating a Docker authentication secret. Alternatively, you can modify the `calico.yaml` file with the images pulled from the alternate host such as `mirror.gcr.io`.

   If you plan to deploy on Ubuntu behind a firewall, run the following commands to enable the firewall:

   ```
   #First add rule to allow ssh so that you can ssh to the VM after the firewall is
   enabled.
   sudo ufw allow ssh

   #Enable firewall
   systemctl enable ufw
   systemctl start ufw
   ```

2. Verify that pods are running, and the cluster is in a `Ready` status:

   ```
   kubectl get pods -A
   kubectl get nodes
   ```

   Next, disable Device Mapper Multipath (DM-Multipath) before installing Longhorn on Ubuntu.

   **CAUTION: Dell Automation Platform does not support multipathing.**

3. Stop the multipath daemon and prevent it from starting at boot:

   **NOTE:** Run these commands on all the three k8s nodes if it is a three-node cluster.

   ```
   sudo systemctl stop multipathd.service multipathd.socket # Stop both service and
   socket for multipathd
   sudo systemctl disable multipathd.service multipathd.socket # Disable both service
   and socket for multipathd
   sudo systemctl mask multipathd.service # Mask the service
   systemctl status multipathd.service # Check the status
   systemctl status multipathd.socket # Check the status
   ```

4. Ensure multipath is not automatically enabled by modifying or removing the /etc/multipath.conf file, or by setting the multipath module to not load:

   ```
   echo "blacklist { devnode \"*\" }" | sudo tee /etc/multipath.conf
   ```

5. Reboot the system.

6. Install the Longhorn block storage provider using the Helm package manager.

   **NOTE:** For more information, see Quick Installation - Install Longhorn on Kubernetes on the Longhorn website. Ensure that the installation requirements are met before installing.

   Dell Automation Platform requires different default Longhorn settings, depending on whether you are configuring on a single-node cluster or a three-node cluster. Using the Helm package manager simplifies this process.

   If installing the Dell Automation Platform orchestrator on a single-node Kubernetes cluster, ensure that the default replica count is set to *1*.

   By default, Longhorn sets a 30% storage reservation on the root disk. The `storageReserved` setting in Longhorn specifies the percentage of disk space that is not allocated to the default disk on each new Longhorn node. This reserved space ensures that there is always some free space available on the disk, which is crucial for system stability and performance.

   a. Add the Longhorn repository to Helm and update the repository:

   ```
   helm repo add longhorn https://charts.longhorn.io
   helm repo update
   ```

   b. Install Longhorn.

   - For single-node Kubernetes:

   ```
   helm install longhorn longhorn/longhorn --namespace longhorn-system
   --createnamespace --set persistence.defaultClassReplicaCount=1 --set
   defaultSettings.defaultReplicaCount=1 --version 1.9.1
   ```

   **CAUTION: Only run this command on a single-node configuration.**

   **NOTE:** When installing Dell Automation Platform orchestrator on a single-node Kubernetes cluster, ensure that the block storage provider sets the default replica count to *1*.

   - For multinode Kubernetes:

   ```
   helm install longhorn longhorn/longhorn --namespace longhorn-system --create-
   namespace --version 1.6.2
   ```

   **CAUTION: Only run this command on a multinode configuration.**

7. Verify that Longhorn pods are running and the cluster is in a `Ready` status:

   ```
   kubectl get pods -A
   kubectl get nodes
   ```

8. Optionally add a node port for accessing the Longhorn UI. This example uses TCP port 31000, but it can be changed to any unused TCP port in the 30000–32767 range.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: v1
   kind: Service
   metadata:
     name: longhorn-nodeport-svc
     namespace: longhorn-system
   spec:
     type: NodePort
     ports:
     - name: http
       nodePort: 31000
       port: 80
       protocol: TCP
       targetPort: http
     selector:
       app: longhorn-ui
   ```

```
  sessionAffinity: None
EOF
```

9. Install haproxy using a helm chart and a values YAML file:
   a. Get the helm chart by running the following commands:

```
helm repo add haproxytech https://haproxytech.github.io/helm-charts
helm repo update
```

   b. Create a YAML file named `haproxy-values.yaml`, with the following content:

```
controller:
  image:
    repository: haproxytech/kubernetes-ingress
    pullPolicy: Always
  imagePullSecrets:
    - name: docker-secret
  service:
    type: LoadBalancer
    externalTrafficPolicy: Local
  config:
    ssl-passthrough: "true"
  hostNetwork: true
  kind: DaemonSet
  defaultTLSSecret:
     enabled: false
```

   **NOTE:** The indentation structure of YAML files is important for correctness and maintainability.

   c. Run the following command to install haproxy:

```
helm install haproxy haproxytech/kubernetes-ingress --namespace haproxy --create-
namespace -f haproxy-values.yaml
```

10. Install metalLB using the manifest.
    Apply the metalLB manifest directly from the repository URL:

```
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.14.8/config/
manifests/metallb-native.yaml
```

11. Add an IPAddressPool configuration for metalLB.
    Dell Automation Platform orchestrator UI and API services use this IPAddressPool. It should be a dedicated IP for Dell Automation Platform orchestrator. On a single-node deployment, it can be the same IP as the host primary interface. See the *Example three-node DNS schema* table in Kubernetes-based deployment environment requirements. Change the `addresses:` value to match the kube-vip IP address:

```
cat <<EOF | kubectl apply -f -
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: ip-pool
  namespace: metallb-system
spec:
  addresses:
  - <desired IPv4 address>/32
EOF
```

12. Install a metrics server by performing the following substeps:
    a. Download the metrics server manifest:

```
curl -LO https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml
```

b. To deploy the metrics server, apply the manifest file to your cluster using kubectl:

```
kubectl apply -f components.yaml
```

c. To verify the deployment, confirm the status of the metrics server pods to ensure that they are running correctly:

```
kubectl get pods -n kube-system
```

d. To test the metrics server, use the kubectl top command to check resource usage:

```
kubectl top nodes

kubectl top pods
```

# Optional configuration of a deployment VM

If there is a requirement to manage the Kubernetes cluster from a remote Linux host, the Dell Automation Platform deployment script can also run on the remote host.

The deployment host requires the following software packages and configurations:

| | |
|---|---|
| **Registry Certificate** | The registry certificate that is added to the local certificate store |
| **Unzip** | For extracting the Dell Automation Platform orchestrator deployment bundle |
| **Docker engine and client** | For managing container images, and pushing them to the registry |
| **kubectl** | To access and manage the target Kubernetes cluster |
| **kubectl configuration file** | For accessing the remote Kubernetes cluster |
| **Helm package manager** | For use by the Dell Automation Platform orchestrator during deployment |

## Configure deployment host prerequisite software

### Steps

1. Verify that the Kubernetes node and registry are resolvable on the network.

   If DNS is leveraged, use the `nslookup` command to verify that IP and hostname are resolvable on the deployment host:

   ```
   nslookup dap-k8s-1.mydomain.edge
   nslookup dap-k8s-2.mydomain.edge
   nslookup dap-k8s-3.mydomain.edge
   nslookup dap-k8s-vip.mydomain.edge
   nslookup registry.mydomain.edge
   ```

   If DNS is not leveraged, update the /etc/hosts on the deployment host:

   ```
   sudo vi /etc/hosts
   192.168.0.51    dap-k8s-1.mydomain.edge
   192.168.0.52    dap-k8s-2.mydomain.edge
   192.168.0.53    dap-k8s-3.mydomain.edge
   192.168.0.54    dap-k8s-vip.mydomain.edge
   192.168.0.100    registry.mydomain.edge
   ```

   These hostnames and FQDNs are examples only. Update according to your environment. Any domain with a suffix of `.local` is a special domain with special semantics. Any FQDN ending in `.local` is `link-local`, which means that the names within this domain are meaningful only on the link where they originate.

   For single-node Kubernetes deployments, only the first node and registry FQDN are needed.

The kube-vip does not initially resolve until the kube-vip static pod is deployed in later steps of this task.

2. Download and add the registry certificate to the certificate store:

```
echo | openssl s_client -showcerts -connect <registryURL>:<port> 2>/dev/null |
openssl x509 > registry.crt
file *.crt
openssl x509 -in registry.crt -text -noout
sudo cp *.crt /usr/local/share/ca-certificates/
sudo update-ca-certificates
```

3. Install the unzip package:

```
sudo apt install unzip
```

4. Install Docker using the official installation script:

```
sudo curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

5. Optionally, add the user to the docker group:

```
sudo usermod -aG docker $USER
```

**NOTE:** The shell session must be reloaded for this change to take effect. Exit and reconnect the session.

6. Install kubectl:

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo gpg --
dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
pkgs.k8s.io/core:/stable:/v1.30/deb/ /' | sudo tee /etc/apt/sources.list.d/
kubernetes.list
sudo apt update
sudo apt install kubectl
```

7. Add the kube configuration file from the Kubernetes cluster to the local user and root user:
   a. On a node from the Kubernetes cluster where kubectl is configured, run the following command to generate a kubectl configuration file:

   ```
   kubectl config view --raw > kubeconfig.yaml
   ```

   b. Copy this configuration YAML to the deployment host:

   ```
   scp kubeconfig.yaml <user>@<deployment host>:/home/<user>
   ```

   c. Copy the kubectl configuration YAML to the default locations for kubectl:

   ```
   mkdir -p $HOME/.kube
   sudo mkdir /root/.kube
   cp kubeconfig.yaml $HOME/.kube/config
   sudo chown $(id -u):$(id -g) $HOME/.kube/config
   sudo cp -i /home/<user>/.kube/config /root/.kube/config
   ```

8. Install the Helm package manager:

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/
get-helm-3
chmod 700 get_helm.sh
sudo ./get_helm.sh --version v3.12.0
```

9. Verify the configuration by running the following commands as sudo. Sudo is required for the Dell Automation Platform orchestrator deployment script. Each command should return without any errors:

```
sudo kubectl get nodes
sudo helm list
sudo docker ps
```

10. Verify that the deployment host can log in to the registry from the Sudo user.

```
sudo docker login <registryURL>
```

The user is prompted for a username and password.

11. Verify that the Docker login is working.

# A

# Pre-deployment checklist

The pre-deployment checklist describes how to verify the Kubernetes installation, node status, pod status, and service status to ensure a successful Dell Automated Platform installation.

**Steps**

1. Verify the Kubernetes installation:

```
kubectl version
Client Version: v1.32.8
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
Server Version: v1.30.14
```

   These commands confirm the client and server versions an ensure that `kubectl` can talk to the cluster.

2. Run the `kubectl get nodes`command:

```
NAME              STATUS   ROLES           AGE     VERSION
sre08229-2ebpqe   Ready    control-plane   5h18m   v1.30.14
```

3. Run the `kubectl get pods -A` command:

```
NAMESPACE          NAME                                                READY   STATUS
   RESTARTS   AGE
calico-apiserver   calico-apiserver-6cd76bf584-7fs2h                   1/1
Running   0          49m
calico-apiserver   calico-apiserver-6cd76bf584-ckvw9                   1/1
Running   0          49m
calico-system      calico-kube-controllers-94885f98f-dghxq             1/1
Running   0          49m
calico-system      calico-node-6nm4h                                   1/1
Running   0          49m
calico-system      calico-typha-78645bb496-4dzgm                       1/1
Running   0          49m
calico-system      csi-node-driver-bktq8                               2/2
Running   0          49m
haproxy            haproxy-kubernetes-ingress-9c844                    1/1
Running   0          49m
kube-system        coredns-55cb58b774-b45j9                            1/1
Running   0          56m
kube-system        coredns-55cb58b774-chf29                            1/1
Running   0          56m
kube-system        etcd-sre08127-f2fybo                                1/1
Running   0          56m
kube-system        kube-apiserver-sre08127-f2fybo                      1/1
Running   0          56m
kube-system        kube-controller-manager-sre08127-f2fybo             1/1
Running   0          56m
kube-system        kube-proxy-mtp4j                                    1/1
Running   0          56m
kube-system        kube-scheduler-sre08127-f2fybo                      1/1
Running   0          56m
kube-system        tigera-operator-76ff79f7fd-q4fq2                    1/1
Running   0          49m
longhorn-system    csi-attacher-698d78f74c-9klcv                       1/1
Running   0          49m
longhorn-system    csi-attacher-698d78f74c-k9b7v                       1/1
Running   0          49m
longhorn-system    csi-attacher-698d78f74c-qjpf4                       1/1
Running   0          49m
longhorn-system    csi-provisioner-67f89896c9-72wlg                    1/1
Running   0          49m
longhorn-system    csi-provisioner-67f89896c9-gx9q8                    1/1
```

```
Running   0          49m
longhorn-system    csi-provisioner-67f89896c9-jpzmc                    1/1
Running   0          49m
longhorn-system    csi-resizer-7966f575f5-dxxw4                        1/1
Running   0          49m
longhorn-system    csi-resizer-7966f575f5-k9zfg                        1/1
Running   0          49m
longhorn-system    csi-resizer-7966f575f5-rcq7s                        1/1
Running   0          49m
longhorn-system    csi-snapshotter-845c75b5b6-gzsrv                    1/1
Running   0          49m
longhorn-system    csi-snapshotter-845c75b5b6-jwnrr                    1/1
Running   0          49m
longhorn-system    csi-snapshotter-845c75b5b6-sc5q7                    1/1
Running   0          49m
longhorn-system    engine-image-ei-b0369a5d-f98mw                      1/1
Running   0          49m
longhorn-system    instance-manager-00f55002e5b03a68f5a9ab070b539d1e   1/1
Running   0          49m
longhorn-system    longhorn-csi-plugin-fqrtm                           3/3
Running   0          49m
longhorn-system    longhorn-driver-deployer-7db66c84c5-46htd           1/1
Running   0          56m
longhorn-system    longhorn-manager-nlgmh                              1/1
Running   0          49m
longhorn-system    longhorn-ui-966cf4bb4-jstxj                         1/1
Running   0          56m
longhorn-system    longhorn-ui-966cf4bb4-wgwkl                         1/1
Running   0          56m
metallb-system     controller-6dd967fdc7-w9fpt                         1/1
Running   0          56m
metallb-system     speaker-7pw2h                                       1/1
Running   0          49m
```

4. Run the `kubectl get svc` command:

```
NAMESPACE          NAME                              TYPE           CLUSTER-IP
EXTERNAL-IP    PORT(S)
  AGE
calico-apiserver   calico-api                        ClusterIP      161.210.97.126
<none>         443/TCP
  54m
calico-system      calico-kube-controllers-metrics   ClusterIP      None
<none>         9094/TCP
  54m
calico-system      calico-typha                      ClusterIP      161.210.33.32
<none>         5473/TCP
  55m
default            kubernetes                        ClusterIP      161.210.0.1
<none>         443/TCP
  61m
haproxy        haproxy-kubernetes-ingress     LoadBalancer   161.210.84.140   172.20.8.127   80:31845/
TCP,443:30540/TCP,443:30540/UDP,1024:32014/TCP,6060:30541/TCP   61m
kube-system        kube-dns                          ClusterIP      161.210.0.10
<none>         53/UDP,53/TCP,9153/TCP
  61m
longhorn-system    longhorn-admission-webhook        ClusterIP      161.210.68.224
<none>         9502/TCP
  61m
longhorn-system    longhorn-backend                  ClusterIP      161.210.67.122
<none>         9500/TCP
  61m
longhorn-system    longhorn-conversion-webhook       ClusterIP      161.210.254.19
<none>         9501/TCP
  61m
longhorn-system    longhorn-engine-manager           ClusterIP      None
<none>         <none>
  61m
longhorn-system    longhorn-frontend                 ClusterIP      161.210.43.197
<none>         80/TCP
  61m
longhorn-system    longhorn-recovery-backend         ClusterIP      161.210.138.88
```

```
<none>          9503/TCP
  61m
longhorn-system    longhorn-replica-manager          ClusterIP      None
<none>          <none>
  61m
metallb-system     metallb-webhook-service           ClusterIP      161.210.65.218
<none>          443/TCP
  61m
```

**NOTE:** You must include an external address, shown in bold above, to successfully deploy the orchestrator.

5. Optionally, describe a pod to get more details using the following format: `kubectl describe pod <pod name> -n <namespace>`.

```
kubectl describe pod controller-6dd967fdc7-w9fpt -n metallb-system
Name:             controller-6dd967fdc7-w9fpt
Namespace:        metallb-system
Priority:         0
Service Account:  controller
Node:             sre08127-f2fybo/172.20.8.127
Start Time:       Wed, 06 Aug 2025 10:32:46 +0000
Labels:           app=metallb
                  component=controller
                  pod-template-hash=6dd967fdc7
Annotations:      cni.projectcalico.org/containerID:
8352c4bac9ef9c4bdfd29d3dfa81472480ff7b533ad37c8ec2cbaaa0e1b7e34b
                  cni.projectcalico.org/podIP: 161.200.12.10/32
                  cni.projectcalico.org/podIPs: 161.200.12.10/32
                  prometheus.io/port: 7472
                  prometheus.io/scrape: true
Status:           Running
IP:               161.200.12.10
IPs:
  IP:           161.200.12.10
Controlled By:  ReplicaSet/controller-6dd967fdc7
Containers:
  controller:
    Container ID:  containerd://
a973912a9a7fa66e6c4a39f9e38dcb1048cd6309e91ce1a6c0ba1190b38ae16c
    Image:         quay.io/metallb/controller:v0.14.8
    Image ID:      quay.io/metallb/
controller@sha256:93b83b39d06bbcb0aedc0eb750c9e43e3c46dc08a6f88400ed96105224d784ec
    Ports:         7472/TCP, 9443/TCP
    Host Ports:    0/TCP, 0/TCP
    Args:
      --port=7472
      --log-level=info
      --tls-min-version=VersionTLS12
    State:          Running
      Started:      Wed, 06 Aug 2025 10:33:14 +0000
    Ready:          True
    Restart Count:  0
    Liveness:       http-get http://:monitoring/metrics delay=10s timeout=1s
period=10s #success=1 #failure=3
    Readiness:      http-get http://:monitoring/metrics delay=10s timeout=1s
period=10s #success=1 #failure=3
    Environment:
      METALLB_ML_SECRET_NAME:  memberlist
      METALLB_DEPLOYMENT:      controller
    Mounts:
      /tmp/k8s-webhook-server/serving-certs from cert (ro)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-blclp (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  cert:
```

```
    Type:        Secret (a volume populated by a Secret)
    SecretName:  metallb-webhook-cert
    Optional:    false
  kube-api-access-blclp:
    Type:                    Projected (a volume that contains injected data from
multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    ConfigMapOptional:       <nil>
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              kubernetes.io/os=linux
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for
300s
Events:
  Type     Reason                  Age                From               Message
  ----     ------                  ----               ----               -------
  Warning  FailedScheduling        59m (x2 over 64m)  default-scheduler  0/1
nodes are available: 1 node(s) had untolerated taint {node.kubernetes.io/not-ready:
}. preemption: 0/1 nodes are available: 1 Preemption is not helpful for scheduling.
  Normal   Scheduled               58m                default-scheduler  Successfully
assigned metallb-system/controller-6dd967fdc7-w9fpt to sre08127-f2fybo
  Warning  FailedCreatePodSandBox  58m                kubelet            Failed
to create pod sandbox: rpc error: code = Unknown desc = failed to setup network for
sandbox "cbf8c47a4fed2f214c79f38a20ab92006a498580e75d3e64db1caaf3ea2f604b": plugin
type="calico" failed (add): stat /var/lib/calico/nodename: no such file or directory:
check that the calico/node container is running and has mounted /var/lib/calico/
  Normal   SandboxChanged          58m (x2 over 58m)  kubelet            Pod sandbox
changed, it will be killed and re-created.
  Normal   Pulling                 58m                kubelet            Pulling
image "quay.io/metallb/controller:v0.14.8"
  Normal   Pulled                  58m                kubelet
   Successfully pulled image "quay.io/metallb/controller:v0.14.8" in 1.735s (11.725s
including waiting). Image size: 27503702 bytes.
  Normal   Created                 58m                kubelet            Created
container: controller
  Normal   Started                 58m                kubelet            Started
container controller
```